CSP Daily Report — Security Audit Report

App: CSP Daily Report (CSPDailyReport-2.0.50+) | **Package:** org.cspdaily.report | **Source version:** 2.0.53 | **Date:** 2026-10-11 | **Reviewer:** Agnes (Kimi Code CLI)

Executive Summary

This report presents the findings of a source-code-level security review of the CSP Daily Report Android application. The audit covered **32 Java source files** across security, data, update, and UI modules.

4 Critical, **4 High**, **4 Medium**, and **4 Low** severity issues were identified. The most urgent concerns are hardcoded cloud credentials, an exposed default super-admin account, and permissive database access policies that collectively allow any reverse-engineered client full read/write access to the production Supabase backend and all CSP operational data.

Finding Summary

Severity	Count	Description
Critical	4	Hardcoded credentials, default admin password, open RLS, plaintext password fallback
High	4	Shared publishable key, no certificate pinning, unvalidated APK URL, unencrypted local DB
Medium	4	Weak biometric binding, lenient timeouts, broad permissions, unencrypted backup export
Low	4	Schema in client, seed PII data, no login rate limit, legacy storage flag
Total: 16 findings across 12 source files		

Detailed Findings

Severity: Critical

F1 · **Critical** · SupabaseClient.java:30-31

Hardcoded Supabase Project URL and Publishable Key

The Supabase project URL and publishable key are defined as `public static final` constants embedded directly in the source code. These values are trivially extractable from any compiled APK using standard reverse-engineering tools (e.g., apktool, JD-GUI).

```
public static final String DEFAULT_URL = "https://sfegesdzmoyolrrbturx.supabase.co";
public static final String DEFAULT_KEY = "sb_publishable_Bvigpve6rd7_Ik57XYzxIw_Q3vi7UMv";
```

Recommendation: Remove hardcoded values. Inject the project URL and key via secure build-time configuration (e.g., Gradle `buildConfigField`) or fetch from a trusted provisioning endpoint at first launch. Rotate the existing key immediately.

F2 · **Critical** · Auth.java:20-36, Auth.java:89

Default Super-Admin Account with Well-Known Password

On first installation the app auto-creates a super-admin user with username `admin` and password `admin123`. The login code explicitly checks for this pair, meaning every fresh install ships with a known-credential administrative account. This is the single highest-impact vulnerability: an attacker with network access and the app can gain super-admin privileges over all data.

```
superAdmin.setPassword("admin123");
superAdmin.isDefaultPassword = true;
// ...
if ("admin".equalsIgnoreCase(user.username) && "admin123".equals(plain)) {
    user.isDefaultPassword = true;
}
```

Recommendation: Remove the auto-created default account entirely. On first launch, require the user to create an account with a strong, randomly-generated initial password. Force a password change before any administrative actions are permitted.

F3 · **Critical** · SupabaseClient.java (embedded SQL schema)

Row Level Security Policies Grant Full Anonymous Access to Sensitive Tables

The SQL schema embedded in the client grants the `anon` role full read/write access (SELECT, INSERT, UPDATE, DELETE) on nearly all tables, including `app_user`, `daily_report`, `followup_log`, and `csp`. With the published key, any attacker can interact with the database as an unauthenticated user.

```
CREATE POLICY "Anon Full Access app_user" ON public.app_user
  FOR ALL TO anon USING (true) WITH CHECK (true);
CREATE POLICY "Anon Full Access daily_report" ON public.daily_report
  FOR ALL TO anon USING (true) WITH CHECK (true);
-- ... repeated for csp, supervisor_csp_map, followup_log, etc.
```

Recommendation: Remove or tighten all `anon` policies on sensitive tables. Require `authenticated` role for all write operations. Implement RLS policies based on the caller's user role (SUPERADMIN, ADMIN, SUPERVISOR). Restrict `app_user` table access to authenticated users only.

F4 · **Critical** · Auth.java:99-109, User.java:96-104

Plaintext Password Fallback for Legacy Users

The authentication logic falls back to plaintext password comparison when a `password_hash` is not present. The `password` column (plaintext) is still read from the database and used for login. If any user record lacks a PBKDF2 hash (e.g., from cloud sync or a migration edge case), the password is stored and compared in cleartext both locally and potentially on the server.

```
if (user.password != null && !user.password.isEmpty()) {
    if (user.password.equals(plain)) {
        user.setPassword(plain); // migrates to hash on next save
        ...
    }
}
```

Recommendation: Enforce PBKDF2 hashing for all accounts at the database level. Reject any login attempt where the stored credential is a plaintext password and force a migration. Remove the plaintext comparison branch entirely after migration is complete.

Severity: High

F5 · High · SupabaseClient.java

Supabase Publishable Key Shared with All Clients

The Supabase `sb_publishable_...` key is distributed to every installed instance. Although technically "public" by design, combined with the permissive RLS policies (F3) this key effectively functions as an unrestricted database credential. Every copy of the APK carries the same effective super-user access to the cloud backend.

Recommendation: Combine with F3 remediation — once RLS is tightened, the publishable key will no longer grant unauthorized access. Consider also rotating keys periodically and using Supabase JWT-based authentication for user-scoped operations.

F6 · High · SupabaseClient.java (all HTTP methods)

No Certificate Pinning on Network Connections

All network communication uses raw `HttpURLConnection` with default trust anchors. On untrusted networks (public Wi-Fi, compromised devices), a MITM attacker can intercept credentials, CSP personal data (names, mobile numbers), and submitted daily reports without detection.

Recommendation: Implement certificate pinning for the Supabase endpoint using Android's `CertificatePinner` class (OkHttp) or a custom `SSLContext`. At minimum, validate the server certificate chain explicitly.

F7 · High · ApkDownloader.java:88

APK Download URL Not Validated Against Expected Host

The downloader accepts any URL returned by the remote config or update metadata without validating the host or scheme. If the remote config is compromised or a MITM attack redirects the update check, a malicious APK could be downloaded and installed (the app already holds `REQUEST_INSTALL_PACKAGES`).

```
URL url = new URL(downloadUrl);
conn = (HttpURLConnection) url.openConnection();
```

Recommendation: White-list the expected update server domain. Reject any download URL whose host does not match the known Supabase project domain. Validate the APK SHA-256 against the expected hash before installation (already partially implemented in `ApkIntegrityVerifier`, but the URL should also be validated first).

F8 · High · Db.java:14

Local SQLite Database Stored Without Encryption

The SQLite database file `csp_daily.db` is stored in the app's private data directory without any encryption. On rooted devices, ADB backup (where supported), or via memory forensics, all stored data — including PBKDF2 password hashes, user PII, and business reports — is readable.

Recommendation: Enable SQLCipher or Android's File-Based Encryption (FBE) for the database. At minimum, use `enableWriteAheadLogging()` and ensure the database file is not included in ADB backups (`allowBackup="false"` is already set correctly).

Severity: Medium

F9 · Medium · LoginScreen.java:324-332

Biometric Sign-In Does Not Bind to Specific User Identity

The 1-touch fingerprint sign-in directly authenticates as the last logged-in user without verifying that the current fingerprint holder is the same person. On a shared device, anyone with an enrolled fingerprint can impersonate the previous user.

Recommendation: Store a biometric binding token (e.g., HMAC of the user ID signed with a device-only key) and verify it during biometric authentication. Alternatively, require password entry after biometric unlock on shared devices.

F10 · Medium · SessionManager.java:133-140

Inactivity Lock Timeout Defaults to 10 Minutes

The default inactivity timeout before locking the session is 600 seconds (10 minutes). For an app handling business-sensitive reporting data, a shorter timeout (e.g., 5 minutes) is more appropriate, especially on shared or unattended devices.

Recommendation: Reduce the default inactivity timeout to 300 seconds. Allow administrators to configure this via `Settings` but enforce a maximum of 600 seconds.

F11 · Medium · AndroidManifest.xml:12-14

Broad Sensitive Permissions Declared

The manifest declares `READ_CALL_LOG` and `READ_PHONE_STATE` in addition to `CALL_PHONE`. The comments suggest these are for follow-up call history tracking, but `READ_PHONE_STATE` is not strictly necessary for that purpose and increases the data exposure surface beyond what is functionally required.

Recommendation: Remove `READ_PHONE_STATE` if not essential. Justify `READ_CALL_LOG` in the Play Store listing and consider replacing with explicit call log queries scoped to the app's own follow-up entries only.

F12 · Medium · Backup.java

Backup/Export Contains Full PII Without Encryption Option

The JSON export includes all CSP names, mobile numbers, branch details, and daily reports. There is no option to encrypt or password-protect the backup file, meaning any shared or stored backup is fully readable.

Recommendation: Add AES-256 encryption support for exported backups with a user-supplied PIN. Store the encryption key derived from the PIN using PBKDF2 and never persist the PIN itself.

Severity: Low

F13 · Low · SupabaseClient.java:915-1144

Full Database Schema Embedded in Client Source

The complete PostgreSQL schema (~230 lines of SQL) is embedded as a Java string constant in the client. While not a direct vulnerability, it provides attackers with complete knowledge of the data model, relationships, and field names, reducing the effort needed to craft targeted API requests once credentials are obtained.

Recommendation: Store the schema SQL separately and reference it from a protected server endpoint or build artifact rather than hardcoding it in the APK.

F14 · Low · Db.java:29-62

Seed Data Contains Real Names and Phone Numbers

Source: Db.java:29-62

The `SEED_CSPS` array contains 30 entries with full real names, mobile numbers, and branch locations. This data should not be bundled in a production APK as it exposes PII of actual field staff and appears in any decompiled source.

Recommendation: Remove all seed CSP data from the source. Populate seed data server-side or via a one-time secure import process after first launch.

F15 · Low · Auth.java:78

No Rate Limiting or Account Lockout on Login Attempts

Both local and Supabase login paths accept unlimited password attempts with no delay, CAPTCHA, or lockout mechanism. An attacker with network access can perform offline brute-force attacks against the PBKDF2 hash (65,536 iterations provides ~100ms per attempt, making ~10 attempts/sec — feasible for short passwords).

Recommendation: Implement exponential backoff between failed attempts (e.g., 1s, 2s, 4s, ... up to 30s). Add a progressive lockout after 5-10 consecutive failures. Increase PBKDF2 iterations to 131,072 or higher for additional resistance.

F16 · **Low** · AndroidManifest.xml:44

Legacy External Storage Flag Enabled

`android:requestLegacyExternalStorage="true"` is set. While harmless on Android 10-12, it prevents the OS from applying scoped storage restrictions and may cause the app to fail Play Store policy checks on newer targeting requirements.

Recommendation: Remove this flag and migrate to scoped storage APIs (`DocumentsContract`, `MediaStore`) for any external storage access.